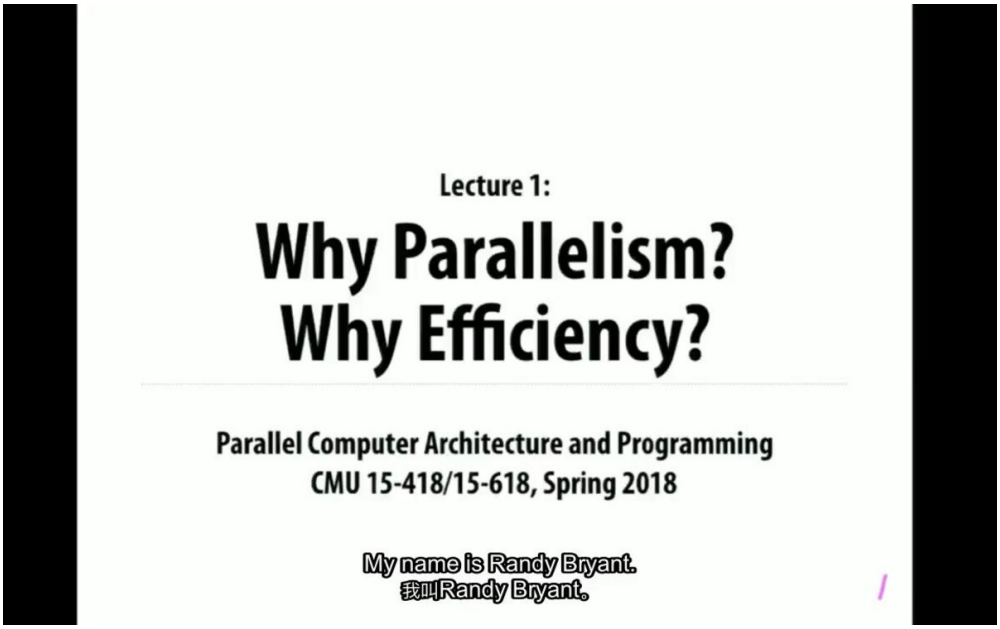
课程笔记

并行计算导论：为什么并行，以及为什么要效率

CMU 15-418/618 并行计算机架构与编程 · Lecture 1（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 16 分 53 秒
视频链接：`https://www.bilibili.com/video/BV1Xz4y1p7ZN/`

目录

1 为什么必须并行：一次被功耗终结的性能曲线 2
1.1 并行机并非新鲜事物：三个阶段的驱动力 2
1.2 2004 年：主频路线为什么走不下去 2
1.3 转折之后的唯一出路 3
1.4 本章小结 4

2 并行的定义：处理单元必须协作，而不是简单叠加 4
2.1 教科书定义与其中的难点 4
2.2 加速比：唯一可比较的性能语言 5
2.3 本章小结 5

3 四场演示：加速为什么总在打折 5
3.1 基准：一个人的 56 秒 5
3.2 两个处理器：34 秒，而不是 28 秒 5
3.3 两个代价的来源：通信与串行收尾 6
3.4 四个处理器：负载不均让结果更差 6
3.5 重新平衡：24 秒与它的代价 7
3.6 全场 150 人：通信拓扑决定归约形态 8
3.7 本章小结 8

4 效率：加速比的另一半真相 8
4.1 效率的定义 9
4.2 同样快两倍，为什么可能是失败 9
4.3 效率为什么成为硬件设计的核心指标 9
4.4 串行成分如何给加速比设上限 10
4.5 本章小结 10

5 这门课怎么教你处理这些代价 11
5.1 性能调试是一个必须经历失败的迭代过程 11
5.2 为什么必须懂一点硬件 11
5.3 每一份作业都是不同的抽象 11
5.4 本章小结 11

6 总结与延伸 11

1　为什么必须并行：一次被功耗终结的性能曲线

这堂课要回答的问题不是”如何写并行程序”，而是更靠前的一步：为什么今天的程序员没有选择余地。答案来自 2004 年前后的一次硬件转折。理解这次转折，才能理解为什么”效率”会成为比”快”更重要的指标。

1.1　并行机并非新鲜事物：三个阶段的驱动力

并行计算机的存在时间比多数读者的编程经历长得多。讲者梳理出一条清晰的驱动力变迁线索，每一阶段的推动者都来自不同的应用领域。

第一个阶段是 1970 年代起的科学计算。当时这类机器被称为”超级计算机”，使用者是希望更快完成数值模拟的研究者。讲者举了三个差异极大的实例：CMU 自己建造的 C.MMP，包含 16 个处理器（视频 00:35:18）；Seymour Cray 在 1980 年代设计的多 CPU 加向量处理超级计算机，处理器数量不多但每一个都极快；以及从 MIT 分出的创业公司 Thinking Machines，路线完全相反——早期机器里放了 65000 个单比特处理器（视频 00:35:57）。讲者对后者的评价带着调侃：它们擅长堆指示灯，很适合科幻电影里”让观众看到计算机在工作”的镜头。

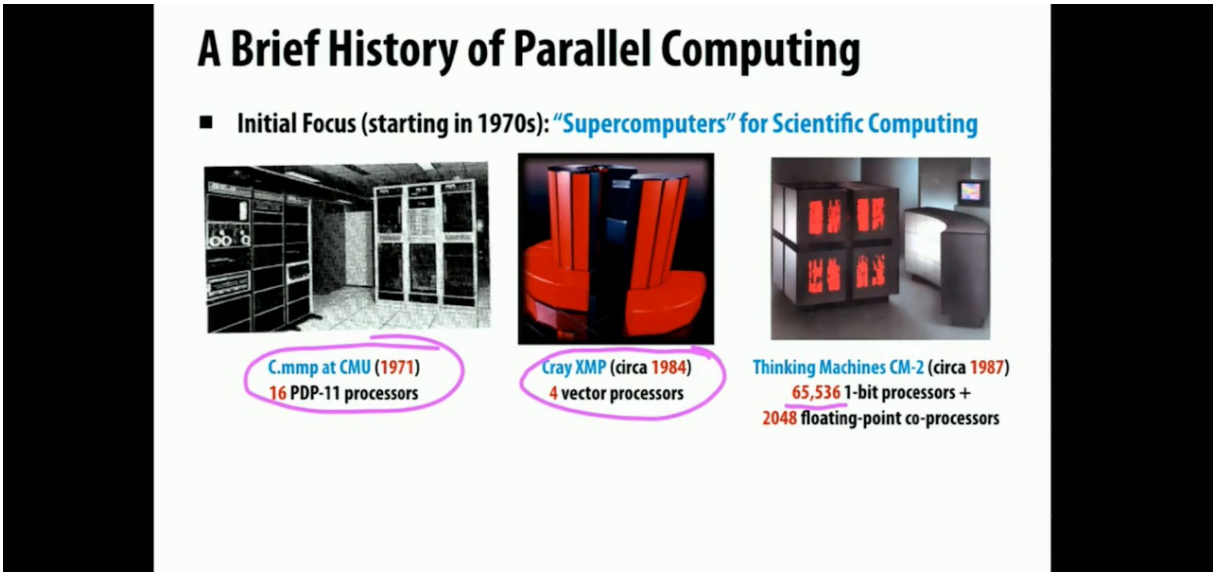


图 1: 讲者的并行计算简史幻灯片，按驱动力阶段划分并行机的应用场景[1]

第二阶段从 1990 年代开始，驱动力换成数据库。数据库研究者发现并行机是加速事务处理的有效手段，于是把关键算法与数据结构散布到多个处理器上。讲者指出这一步的意义在于市场规模的转换：数据库的市场大于科学计算，并行机因此从研究设备变成了一门生意。Oracle 与 Sun 都在卖这类机器。

第三阶段就是今天读者熟悉的形态——通用多核处理器。这一阶段并非由应用需求推动，而是被下一小节的硬件困境逼出来的。

1.2　2004 年：主频路线为什么走不下去

在很长一段时间里，处理器性能呈现稳定的指数增长，讲者的估算是大约每三年翻一倍（视频 00:38:15）。驱动这条曲线的因素有好几个：数据通路不断加宽；硬件会向前扫描指令流，找出互不

[1] 视频画面时间区间：00:36:15–00:36:30。若为裁剪图，时间区间仍对应原视频画面。

依赖的指令并同时发射（讲者提到最高四路）；以及最主要的一项——时钟频率的指数提升。

时钟频率的上涨带来了一个副作用，讲者用一张 Intel 内部的功耗密度幻灯片把这个问题讲得很具体。功耗密度指芯片表面单位面积产生多少瓦特，数值越高，散热压力越大，否则芯片会因过热而失效。早期 Intel 处理器只有几瓦，Pentium 已经热到可以”煎鸡蛋”、温度接近电热板。当时的 Intel CTO 在技术会议上顺势推演：如果主频继续指数上涨，芯片表面温度将依次达到核反应堆内部、火箭喷嘴，最终抵达太阳表面的量级。

这个推演并非玩笑。讲者提到 2004 年 5 月 17 日《纽约时报》的报道：Intel 取消了两个旗舰项目，原因是无法在不让芯片熔化的前提下解决散热问题。他在当时正供职于 Intel，因此把这一事件描述为”一夜之间世界变了”。值得注意的是这里有一个判断失误的代价：Intel 在 1990 年代的大规模营销中教会了消费者”性能等于主频”，这让它自己也难以轻易放弃这条路线。

把主频当成性能的同义词

主频只是性能的一个因子。同一主频下，不同架构的实际吞吐可以相差数倍。2004 年之后，这条等价关系彻底失效，但在消费市场的宣传语境中它长期残留。判断一颗处理器的性能时，主频必须与架构、核心数、缓存层次一起看。

1.3 转折之后的唯一出路

主频停滞之后，硬件设计师能做的是把多个核心放进同一颗芯片。这解释了为什么今天的手机、笔记本、服务器都提供了越来越多的并行硬件。讲者的结论很直接：单线程性能不预期会再变快，所以一切都落在并行上。

这个结论对读者意味着责任转移。过去性能提升由硬件厂商自动交付，软件无需改动就能变快；现在硬件只提供更多并行单元，能否把这些单元变成实际速度，取决于软件是否被正确地并行化。讲者在课程末尾把这一点称为”秘密武器”——走出这门课的人会比别人更懂这些约束。

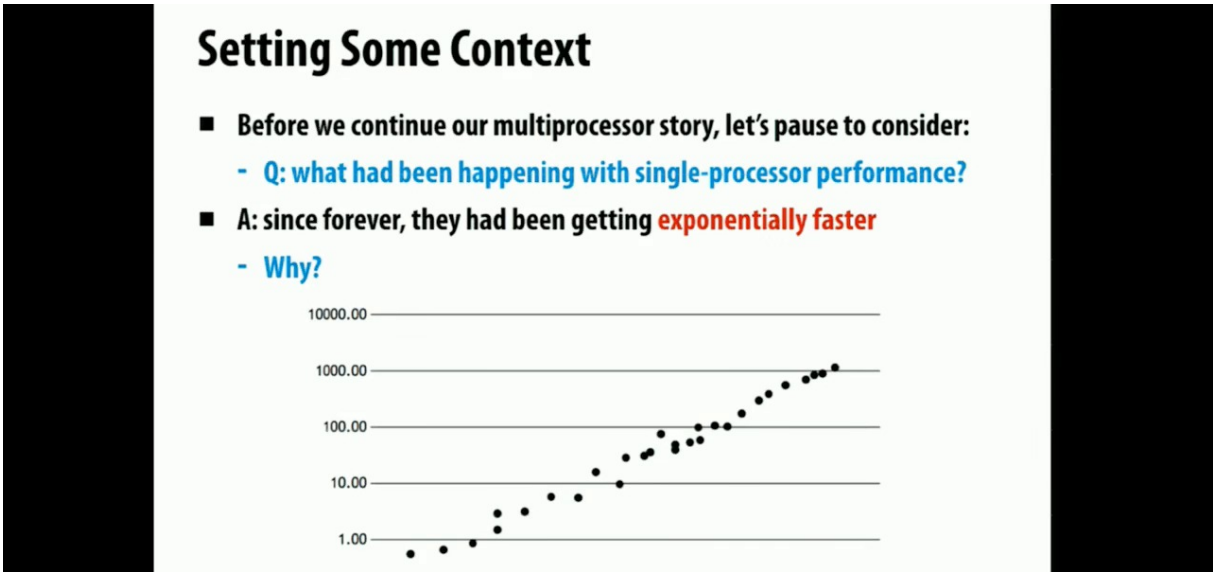


图 2: 讲者插入的背景页，用于回顾单处理器性能曾经如何指数增长[2]

[2]视频画面时间区间：00:38:25–00:38:40。若为裁剪图，时间区间仍对应原视频画面。

1.4 本章小结

性能增长的来源已经从”更快的单核”切换到”更多的核心”，而这次切换是被功耗密度逼出来的，不是设计偏好。并行硬件因此成为默认前提：从 Intel 取消主频路线的 2004 年算起，软件若不并行就无法再获得硬件红利。

有了硬件，接下来的问题是它能否兑现。下一章给出并行的定义，并说明兑现过程为什么一再打折。

2 并行的定义：处理单元必须协作，而不是简单叠加

如果并行只是”装更多处理器”，性能就会随核心数线性增长。实际并非如此，原因在于定义里一个容易被忽略的动词。

2.1 教科书定义与其中的难点

讲者给出了一份”教科书式”的定义：并行计算机是一组处理单元，它们协同起来快速解决问题（视频 00:50:00）。定义包含两个要件，第二个才是关键。处理单元的存在是显而易见的，但让这些单元”协同”去解决同一个问题，才是这门课的全部难度所在。讲者的原话是：它远不只是拥有一堆处理器。

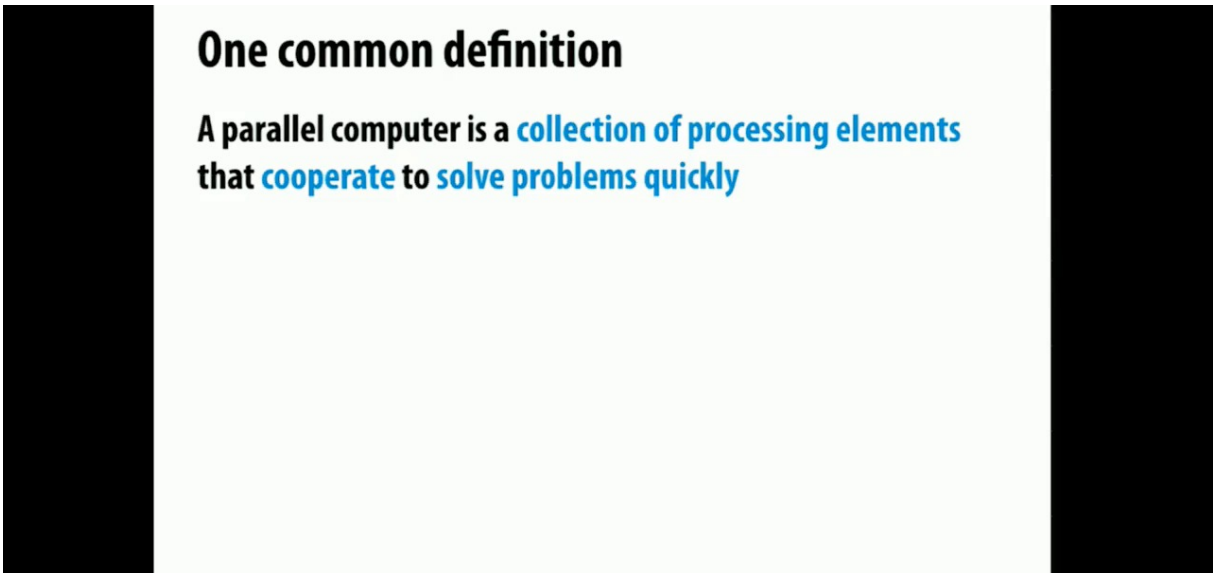


图 3: 并行计算机的定义幻灯片，强调处理单元之间必须协作[3]

并行 ≠ 处理器数量

判定一台机器是否在并行工作，看的是多个处理单元是否在协作求解同一个问题。各自独立处理不同任务属于并发，不是并行。这个区分决定了性能指标该按什么问题单位去度量。

定义给出之后，需要一个能把”协作效果”量化的工具。讲者拒绝使用”快了百分之五十”这类说法，因为”快 100%”究竟指耗时减半还是耗时归零会产生歧义。替代方案是加速比。

[3]视频画面时间区间：00:50:08–00:50:23。若为裁剪图，时间区间仍对应原视频画面。

2.2 加速比：唯一可比较的性能语言

加速比衡量同一份工作在单处理器上的耗时与在多处理器上的耗时之比：

$$S(n) = \frac{T(1)}{T(n)}$$

- $S(n)$：使用 n 个处理单元时的加速比；
- $T(1)$：同一计算在单个处理单元上的耗时；
- $T(n)$：同一计算在 n 个处理单元上的耗时。

这个比值把”变快”从模糊的形容词变成可比的数量。讲者强调的关键之处在于分母是同一份工作，而不是同一台机器上的任意两次运行——否则加速比会被问题规模或实现版本的差异污染。

有了可测量的量，下一个问题自然浮现：处理器数量翻倍，加速比会不会也翻倍。讲者用一场课堂演示给出了否定答案。

2.3 本章小结

并行的定义要求处理单元协作解决同一个问题，协作本身是成本来源。加速比 $S(n) = T(1)/T(n)$ 是衡量协作效果的唯一可比较指标，它衡量的始终是同一份工作。

定义了指标之后，理论预期随之明确：若 n 个处理器完全有效，应有 $S(n) = n$。下面的演示用来检验这个预期，结果每次都不达标，而每次不达标的原因各不相同。

3 四场演示：加速为什么总在打折

这一章的核心材料是讲者组织的课堂演示。他请学生上台，只用纸笔传话、不许互相交谈，把”处理器之间必须通过共享介质通信”这一硬件约束变成可观察的行为。演示从 1 个处理器开始逐步扩展到全场约 150 人（视频 01:07:49）。四个数据点构成一条不理想的扩展曲线。

3.1 基准：一个人的 56 秒

第一位志愿者 Sam 需要在脑中把 16 个一位数相加。计时结果是 56 秒（视频 00:53:37）。这个数字确立了 $T(1)$ 的基准。讲者特意让 Sam 用心算而非纸笔，并说明这一轮的意义只是拿到参照点。

3.2 两个处理器：34 秒，而不是 28 秒

第二轮由两位志愿者分别计算一半的数字，最终需要合并结果。耗时降到 34 秒（视频 00:56:09）。用加速比表达：

$$S(2) = \frac{56}{34} \approx 1.65$$

- 分子 56：单处理器基准耗时，单位为秒；
- 分母 34：两处理器耗时，单位为秒；
- 结果 1.65：实际加速比，低于理论上限 2。

处理器数量翻倍，加速比不到 1.65。讲者让学生自己寻找原因，课堂给出的解释归结为两点，他都借演示设计落实了。

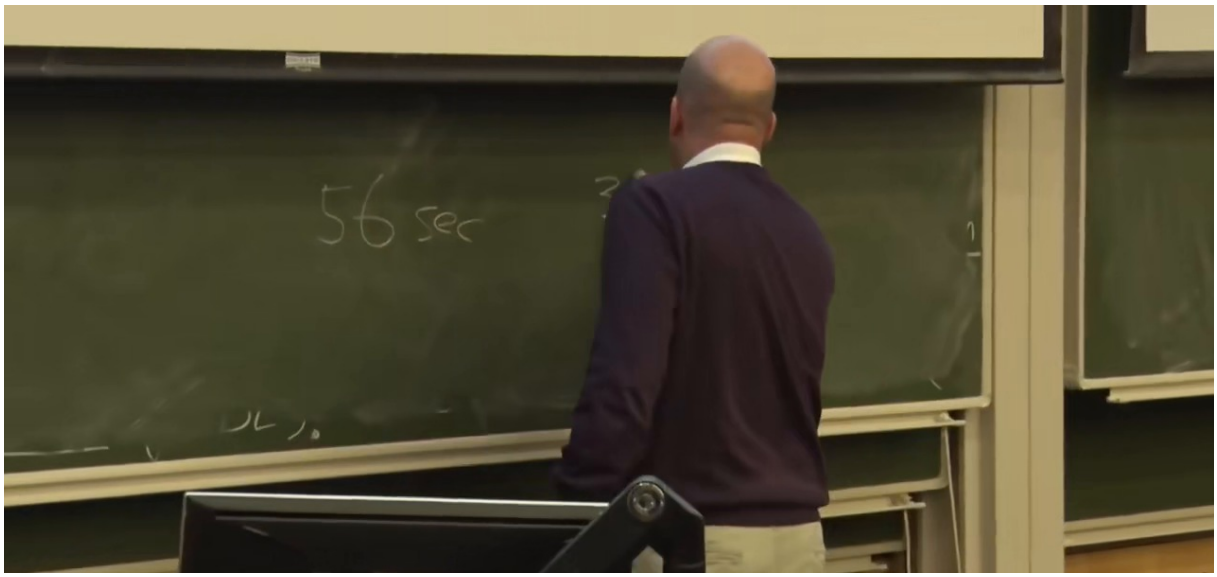


图 4: 两处理器演示结束后的计时幻灯片，记录 56 秒与 34 秒两个数字[4]

3.3 两个代价的来源：通信与串行收尾

第一个代价是通信。讲者故意让其中一位志愿者 Frank 从教室后排走到前排。这不是舞台调度，而是模拟真实硬件的行为：即使多个核心在同一颗芯片上，它们交换数据也必须经过连线，而信号传输需要时间。把 Frank 的行走时间换成物理量，就是互连延迟。

第二个代价是收尾。Frank 走到前面时，搭档其实还没有算完。他必须等对方的结果写下来，读出纸上的数字，再做一次加法，才能得到总数。这段”等待加最终合并”的时间无法被并行化，它是串行成分。

为什么用人类演示讲硬件

人对”走过去看一眼再算”的耗时是有直觉的，而 CPU 核心之间的数据传递抽象掉了这种身体感。讲者用志愿者替代核心，把抽象延迟变成可看见的几秒钟。类比在讲到缓存一致性、流水线细节时会失效，但用于说明”通信有成本、归约需等待”是精确的。

3.4 四个处理器：负载不均让结果更差

按线性预期，4 个处理器应带来接近 4 倍的加速。实际耗时是 39 秒（视频 01:01:49），加速比只有约 1.44：

$$S(4)_{\text{unbalanced}} = \frac{56}{39} \approx 1.44$$

这比两个处理器的 1.65 还差。处理器数量增加了，性能反而下降。讲者在这里设置了本轮演示的关键变量：他在分配数字时故意让一位志愿者拿到了远超其余三人的工作量。学生很快指出这位志愿者分到的数字”远多于四个”。

[4]视频画面时间区间：00:56:12–00:56:27。若为裁剪图，时间区间仍对应原视频画面。

讲者随后解释了为什么这样设计。在真实并行程序中，把工作量均匀切开并不总是容易做到，而负载不均的后果是可以预测的：完成得快的处理器进入空闲，却无法提前结束整个计算，因为最终结果必须等最后一个处理器交卷。三人在等待，一人在计算，这段时间里三个核心的资源被完全浪费。

增加核心数反而变慢

4 个处理器耗时 39 秒，劣于 2 个处理器的 34 秒。这不是测量误差，而是负载不均的直接后果。加速比的分母取决于最慢的那个处理单元，而不是平均值。评估并行方案时，只看总工作量是否被切分是不够的，必须看切分后最大分片的大小。

3.5 重新平衡：24 秒与它的代价

讲者允许四位志愿者在开始前短暂协商策略，之后仍禁止交谈，但可以移动数字。他们的做法是把数字重新分配成每人四个。耗时降到 24 秒（视频 01:06:21），加速比约为：

$$S(4)_{\text{balanced}} = \frac{56}{24} \approx 2.33$$

同一个 4 处理器配置，从 1.44 提升到 2.33，唯一的改动是负载分布。讲者同时提醒这项改进并非免费：重新平衡本身占用了时间，属于额外开销。在这个例子里开销远小于收益，但在实际系统中，动态负载均衡的调度成本可能吃掉全部收益。



图 5: 四处理器负载不均的一轮，计时板同时显示此前两轮的 34 秒与本轮的 39 秒[5]

[5] 视频画面时间区间：01:02:00–01:02:15。若为裁剪图，时间区间仍对应原视频画面。



图 6: 重新平衡后的一轮，计时板并列展示三轮结果以对比加速效果[6]

3.6 全场 150 人：通信拓扑决定归约形态

最后的演示把规模推到极限。全教室约 150 名学生每人报出自己选修的课程门数，规则是只能与前后左右相邻的人通信，不许隔空喊话（视频 01:07:49）。这相当于一台通信拓扑受限于近邻连接的并行机。

学生提出的方案体现了两种结构。第一种是先将同一排的数字横向相加，再把各排结果纵向汇总到前排；第二种是让求和从两端同时向中间推进，使并行度更高。讲者认可这些思路，因为它们在限制条件下减少了必须串行经过的步骤数。

这一轮演示的要点不是耗时数字，而是揭示了一条规律：当通信只能发生在近邻之间时，求和无法一步完成，必须组织成树形或多级归约，通信层数随规模增长。这正是真实并行机上大规模归约的成本来源。

3.7 本章小结

四个数据点构成一条不达标的扩展曲线：$S(2) \approx 1.65$、$S(4) \approx 1.44$、重平衡后 $S(4) \approx 2.33$；规模扩展到全场后，瓶颈转为通信层数。三种代价分别是通信延迟、串行收尾、负载不均，第四种代价隐藏在前三者之中——为了消除它们所做的协调本身要花时间。

演示回答的是” 加速比为什么小于核心数”。它没有回答另一件事：当加速比小于核心数时，这些多余的处理器是浪费还是仍有价值。这取决于如何衡量成本，是下一章的主题。

4 效率：加速比的另一半真相

加速比只回答” 快了多少”，不回答” 值不值”。同样一个加速比，在不同成本结构下可以是一次成功，也可以是一次失败。

[6] 视频画面时间区间：01:05:40–01:05:55。若为裁剪图，时间区间仍对应原视频画面。

4.1 效率的定义

效率把加速比放到处理器数量上归一化：

$$E(n) = \frac{S(n)}{n}$$

- $E(n)$：使用 n 个处理单元时的并行效率，取值在 0 到 1 之间；
- $S(n)$：同一计算在 n 个处理单元上的加速比；
- n：投入的处理单元数量。

$E(n) = 1$ 表示每个处理器都物尽其用，$E(n) = 0.5$ 表示有一半的算力被浪费。用演示数据代入：$E(2) = 1.65/2 \approx 0.83$，负载不均的 $E(4) = 1.44/4 = 0.36$，重新平衡后 $E(4) = 2.33/4 \approx 0.58$。同一份计算，在四个不同的组织方式下效率相差两倍以上。

4.2 同样快两倍，为什么可能是失败

讲者用一道情境题把效率的意义讲透：如果你在实习时把一个问题放到 10 个处理器上，只快了 2 倍，老板会给你升职还是解雇你？他没有立刻给出答案，而是指出结果取决于资源是否稀缺。

如果这 10 个处理器本来就在你手里——比如手机里的 GPU，无论你是否使用它都在耗电和占面积——那么把原本达不到实时帧率的软件加速到实时，就是相当大的成功。反之，若你按 10 台机器在云上租用计费，却只换来很小的收益，这次并行化的投入产出比就很差。

效率取决于边际成本

并行是否值得，不能只看加速比。判断依据是新增的处理单元是否本来就会存在：既有的、已经付费的资源上获得加速是净收益；为并行新增的按量计费资源上只获得小加速则是净损失。同一组数字在两种情境下结论相反。

4.3 效率为什么成为硬件设计的核心指标

效率的重要性不只对软件成立。讲者指出 2004 年之前，硬件设计的目标是” 在一颗芯片上放一个尽可能快的处理器”，为了榨取最后一点性能，设计变得极度投机且低效。而这种设计在当时是正确的——因为芯片面积只能容纳一个核心，不用掉就浪费了。

多个核心进入芯片之后，判断标准翻转了。每个核心的面积决定了能塞进多少个核心，因此” 每单位面积的性能” 成为关键指标；同时，服务器机房的供电能力有上限，移动设备的电池容量有上限，于是” 每瓦性能” 同样关键。讲者把这种从追求绝对速度到追求单位资源产出的转变，视为 2004 年之后硬件设计的根本变化。

维度	2004 年之前（单核路线）	2004 年之后（多核路线）
设计目标	单个处理器尽可能快	单位面积与单位能耗的性能
主要手段	提高主频、投机执行	增加核心数、异构与专用单元
是否宽容低效	宽容，面积只能用在一处	不宽容，浪费面积就少放核心
性能增长来源	硬件自动交付	软件必须并行化才能兑现

表 1: 硬件设计范式的转变及其对软件的后果

4.4 串行成分如何给加速比设上限

演示已经用具体数字展示了加速比低于核心数，讲者没有在黑板上推导公式，但给出了足够的因果结构，可以据此整理出一条硬性约束。

设一份计算中不可并行的比例记为 f，剩余 $1-f$ 部分可以完全并行。若忽略通信与调度开销，n 个处理器下的耗时至少包含那部分串行工作：

$$S(n) \le \frac{1}{f + \dfrac{1-f}{n}}$$

- f：计算中串行成分占总工作量的比例，$0 \le f \le 1$；
- n：处理单元数量；
- 当 $n \to \infty$ 时，上界趋近于 $1/f$。

这个关系解释了演示中”两个处理器只得到 1.65 倍”的现象：即使并行部分完美分摊，Frank 走到台前与最终核对的那一段时间仍然串行。它也解释了为什么不加区分地增加核心会迅速失去意义——上界由 f 决定，与 n 无关。本讲义据此整理为推论，讲者在本课中并未写出该式。

把加速比当成扩张性能的万能钥匙

若一份计算中有 10% 无法并行，则无论投入多少处理器，加速比都不会超过 10 倍。在资源规划阶段先估算串行成分，比事后增加机器更有效。

4.5 本章小结

效率 $E(n) = S(n)/n$ 把加速比换算为资源利用率，它能区分”既得资源上的提速”和”新增付费资源上的小提速”这两种性质相反的处境。2004 年后的硬件设计也转向同一标准：每单位面积与每瓦的性能。串行成分 f 为加速比设定与 n 无关的上界，这是”无限加核”思路的硬性天花板。

到这里，代价的种类已经清楚：通信、负载不均、串行成分、协调开销。剩下一个疑问：这些代价能否在写代码之前被预测，从而避免反复试错？讲者的回答是不能，这正是下一章要说明的课程方法。

5 这门课怎么教你处理这些代价

前面几章建立了并行的动机与代价结构。这一章转向方法论：面对这些代价，工程上应该采取什么样的工作方式。

5.1 性能调试是一个必须经历失败的迭代过程

讲者对课程方法的说明与多数课程不同。他明确表示，这门课关心的是性能调试与调优，而不仅是正确性。与只要求正确结果的课程相比，这里的评分标准包含了过程：他会认真阅读学生记录如何尝试、为何失败、如何推理下一步的写作。

讲者把这一点提升为课程的核心命题：这门课讲的是优化的过程，而不是优化后的那个数字。理由是系统复杂到没有人能在第一次就给出完美方案。真实世界的做法是做出最好的估计，形成策略，尝试，观察结果，修正判断。他在课程开头就要求学生在作业中完整记录这个过程。

为什么过程比结果更值得记录

并行系统的性能由多项相互牵制的因素共同决定：通信、负载均衡、缓存局部性、同步开销。这些因素之间的权衡关系因机器而异，无法从教科书条件直接推出。记录失败尝试的行为本身就是知识的积累方式——它保留了”在什么条件下某策略失效”这一无法从最终代码里读出的信息。

5.2 为什么必须懂一点硬件

讲者给出的第二个方法要点是硬件知识不可跳过。即使读者完全不打算设计硬件，也必须了解硬件层面的约束，原因是对比序列程序时的一个本质差异：在串行程序里，硬件的内部细节对性能的影响可以被很大程度地忽略，而在并行程序里，这些细节直接决定性能。

他举的例子是瓶颈。哪一部分限制了吞吐，往往取决于缓存层次、互连结构、同步原语的实现方式，这些都不是编程语言层面能表达的信息。不了解它们，性能问题就只能靠猜。

5.3 每一份作业都是不同的抽象

课程的第三个特点是抽象层次的多样性。讲者提到，每份作业对应一套完全不同的并行思维抽象。这意味着不存在一个统一的并行编程模型能覆盖所有场景，读者的能力目标是学会在给定抽象下推理开销，而不是记住某个库的调用方式。

5.4 本章小结

课程方法有三个支点：把性能调优当作迭代过程并把失败记录为知识；把硬件瓶颈当作必须理解的约束而非黑盒；把不同并行抽象当作需要分别掌握的工具。这三点的共同前提是前面建立的代价结构——正因为通信、负载、串行成分无法事先完全预测，才需要这种以实验为驱动的工程方法。

6 总结与延伸

这堂导论课没有给出任何并行算法，它给出的是一套判断框架。

讲者的收尾结论可以压缩为一句话：单线程性能不预期会再变快，所以一切都落在并行上，而这件事并不容易做。他把难度转化为一种优势——理解这些约束的人在进入行业时会比同龄人更清楚瓶颈在哪里。

把全课串起来看，逻辑链条是完整的。2004 年功耗密度终结了主频路线，硬件被迫转向多核，性能红利从此需要软件兑现。兑现的前提是处理单元之间的协作，而协作必然产生通信、串行收尾与协调开销。四场演示用 56 秒、34 秒、39 秒、24 秒四个数字把抽象代价变成了可观察的事实：两个处理器只得到约 1.65 倍，四个处理器在负载不均时甚至不如两个，重新平衡后才回升到约 2.33 倍。效率指标进一步说明，这些折扣是否可接受取决于资源是否已经付费；串行成分则为任何加速设定了与核心数无关的上界。

以下三点是本讲义在讲者材料之上做的压缩，需要与讲者原话区分开：

- 加速比与效率必须成对使用。只看加速比会误判一个浪费了七成算力的方案为成功，只看效率会误判一个在既有资源上获得真实提速的方案为失败。
- 演示中的三种代价并不独立。负载不均会诱导出额外的协调开销，协调开销本身又是串行成分的一部分。把它们当成可分别优化的独立项，会在实践中反复遇到此消彼长。
- 讲者的方法论指向一个可操作的优先级：先测量，再估算串行成分与最大分片，最后才考虑增加处理器。顺序颠倒时，硬件投入的收益会被负载与通信问题吞掉。

需要说明这堂课的边界。演示只覆盖了求和一个计算形态，通信被简化为” 走一段路” 和” 看一眼纸”，没有涉及缓存一致性、同步原语竞争、伪共享等真实机制；讲者也明确表示后四到五次课才会讨论这些问题与并行软件的关系。因此本文给出的是判断框架而非调优技术。

一个具体的下一步问题可以直接从演示数据提出：4 个处理器重新平衡后达到 24 秒，而理论下限是 14 秒（56 除以 4）。差的这 10 秒里，有多少属于重新平衡的一次性开销，有多少属于每轮都要付出的归约成本？区分这两者，决定了该优化调度策略还是该改换归约结构。